DSC630 10.2 Assignment Ayden Trusty

March 1, 2026

1 10.2 Assignment: Recommender System

1.1 DSC630 Ayden Trusty

;========== */ Title: Assignment 10.2 Author(s): Abbott, Dean. Applied Predictive Analytics (Textbook) Date: 28 February 2026 Modified By: Ayden Trusty Description: This program is to help me understand and practice using Python and/or R to build a recommendation system.

[2]:
```
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity
```

[4]:
```
# Load data
ratings = pd.read_csv("ratings.csv")
movies = pd.read_csv("movies.csv")
```

[14]:
```
ratings.shape
```

[14]:
```
(100836, 4)
```

[12]:
```
ratings.head()
```

[12]:

	userId	movieId	rating	timestamp
0	1	1	4.0	964982703
1	1	3	4.0	964981247
2	1	6	4.0	964982224
3	1	47	5.0	964983815
4	1	50	5.0	964982931

[15]:
```
movies.shape
```

[15]:
```
(9742, 3)
```

[13]:
```
movies.head()
```

[13]:

	movieId	title	\
0	1	Toy Story (1995)	
1	2	Jumanji (1995)	
2	3	Grumpier Old Men (1995)	

```
3        4            Waiting to Exhale (1995)
4        5  Father of the Bride Part II (1995)

                                        genres
0  Adventure|Animation|Children|Comedy|Fantasy
1                   Adventure|Children|Fantasy
2                              Comedy|Romance
3                        Comedy|Drama|Romance
4                                      Comedy
```

Not sure why, but I almost forgot what to do with multiple datasets.

[5]:
```
# Merge datasets
data = pd.merge(ratings, movies, on='movieId')
```

[16]:
```
data.shape
```

[16]:
```
(100836, 6)
```

[17]:
```
data.head()
```

[17]:
```
   userId  movieId  rating   timestamp             title  \
0       1        1     4.0   964982703  Toy Story (1995)
1       5        1     4.0   847434962  Toy Story (1995)
2       7        1     4.5  1106635946  Toy Story (1995)
3      15        1     2.5  1510577970  Toy Story (1995)
4      17        1     4.5  1305696483  Toy Story (1995)

                                        genres
0  Adventure|Animation|Children|Comedy|Fantasy
1  Adventure|Animation|Children|Comedy|Fantasy
2  Adventure|Animation|Children|Comedy|Fantasy
3  Adventure|Animation|Children|Comedy|Fantasy
4  Adventure|Animation|Children|Comedy|Fantasy
```

[19]:
```
data.isna().sum()
```

[19]:
```
userId       0
movieId      0
rating       0
timestamp    0
title        0
genres       0
dtype: int64
```

Just checking everything out and making sure I don't run into something later on.

[6]:
```
# Create user-movie matrix
user_movie_matrix = data.pivot_table(
    index='userId',
    columns='title',
    values='rating'
)
```

Matrix done, now just need to make sure that anything missing/na is set to 0 so everything plays nice.

[7]:
```
# Fill missing values with 0
user_movie_matrix_filled = user_movie_matrix.fillna(0)
```

[ ]:
```
Now I need to actually perform the calculation/have everything assigned a␣
↪number.
```

[8]:
```
# Compute cosine similarity between movies
movie_similarity = cosine_similarity(user_movie_matrix_filled.T)
```

[9]:
```
# Convert to DataFrame
movie_similarity_df = pd.DataFrame(
    movie_similarity,
    index=user_movie_matrix.columns,
    columns=user_movie_matrix.columns
)
```

[ ]:
```
Function time.
```

[10]:
```
def recommend_movies(movie_title, num_recommendations=10):
    # Make sure there's a response for troubleshooting
    if movie_title not in movie_similarity_df.columns:
        return "Movie not found in dataset."

    similarity_scores = movie_similarity_df[movie_title]
    similar_movies = similarity_scores.sort_values(ascending=False)

    # Remove the movie itself to prevent interference
    similar_movies = similar_movies.iloc[1:num_recommendations+1]

    return similar_movies.index.tolist()
```

[11]:
```
# Example usage
recommend_movies("Toy Story (1995)")
```

[11]:
```
['Toy Story 2 (1999)',
 'Jurassic Park (1993)',
 'Independence Day (a.k.a. ID4) (1996)',
 'Star Wars: Episode IV - A New Hope (1977)',
```

```
 'Forrest Gump (1994)',
 'Lion King, The (1994)',
 'Star Wars: Episode VI - Return of the Jedi (1983)',
 'Mission: Impossible (1996)',
 'Groundhog Day (1993)',
 'Back to the Future (1985)']
```

1.2 Write-up

I began by loading the ratings and movies files, merging them on movieId, and creating a user–item matrix where rows represent users, columns represent movie titles, and values represent ratings. Because not every user rated every movie, I filled missing values with zeros to allow similarity calculations. I then transposed the matrix and used cosine similarity to compute how similar movies are to one another based on shared rating patterns across users. After generating a movie-to-movie similarity matrix, I implemented a recommendation function that accepts a movie title as input, sorts all other movies by similarity score, excludes the selected movie, and returns the top ten most similar films. This approach leverages collective user behavior to recommend movies without requiring demographic data, though it is limited by sparsity.

[ ]: